

AI & Machine Learning - Task 3

Model Validation, Overfitting Control & Hyperparameter Tuning

1. Introduction

This project focuses on validating machine learning regression models using the California Housing Dataset. The objective is to compare multiple regression algorithms, identify overfitting, apply cross-validation, and improve model performance through hyperparameter tuning.

2. Dataset Description

The California Housing Dataset from scikit-learn contains information such as median income, house age, average rooms, average bedrooms, population, average occupancy, latitude, and longitude. The target variable is the median house price.

3. Data Preprocessing

The dataset was explored using Pandas. Missing values were checked, features and target were separated, and StandardScaler was applied for feature scaling. The dataset was then divided into training (80%) and testing (20%) sets.

Model Development and Validation

4. Models Implemented

Three regression models were trained: Linear Regression, Ridge Regression, and Decision Tree Regressor. Linear Regression was used as the baseline model, Ridge Regression reduced overfitting through L2 regularization, and Decision Tree captured nonlinear relationships.

5. Overfitting Analysis

The Decision Tree produced a very low training error but a higher testing error, indicating overfitting. This showed that the model memorized the training data instead of generalizing well to unseen data.

6. Cross-Validation

Five-fold cross-validation was performed using `cross_val_score()`. This technique evaluates the model on multiple data splits, providing a more reliable estimate of its performance than a single train-test split.

7. Hyperparameter Tuning

`GridSearchCV` was used to optimize the Decision Tree by searching for the best values of `max_depth` and `min_samples_split`. The best estimator was selected automatically based on cross-validation performance.

Result & Conclusion

8. Evaluation Metrics

Model performance was measured using RMSE (Root Mean Squared Error) and R^2 Score.

Lower RMSE indicates smaller prediction errors, while higher R^2 indicates better predictive performance.

9. Model Comparison

The results of Linear Regression, Ridge Regression, and the tuned Decision Tree were compared.

The optimized Decision Tree demonstrated improved generalization after hyperparameter tuning.

10. Conclusion

This project demonstrated the complete workflow of machine learning model validation. Overfitting was successfully detected and reduced using GridSearchCV and cross-validation. These techniques improve model reliability and are essential before deploying machine learning models in real-world applications.

Tools Used:

Python, Pandas, NumPy, Matplotlib, Scikit-learn, Jupyter Notebook.

Outcome:

Successfully completed AI & Machine Learning Task 3 with model validation, overfitting analysis, cross-validation, hyperparameter tuning, visualization, and model comparison.

AI_ML_Task3_Model_Validation_Tuning.ipynb > param_grid = {

```
train_pred = tree.predict(x_train)
```

```
test_pred = tree.predict(x_test)
```

[21] ✓ 0.0s

```
train_rmse = np.sqrt(mean_squared_error(y_train, train_pred))
```

```
test_rmse = np.sqrt(mean_squared_error(y_test, test_pred))
```

```
print("Training RMSE :", train_rmse)
```

```
print("Testing RMSE :", test_rmse)
```

[22] ✓ 0.0s

... Training RMSE : 3.218325866275131e-16
Testing RMSE : 0.7030445773467542

```
cv_scores = cross_val_score(  
    tree,  
    x_scaled,  
    y,  
    cv=5,  
    scoring="neg_root_mean_squared_error"  
)
```

```
cv_rmse = -cv_scores.mean()
```

```
print(cv_scores)
```

```
print(cv_rmse)
```

[46] ✓ 0.6s

... [-0.88767617 -0.82781619 -0.89845112 -0.94507041 -0.91950207]
0.8957031908951016

```
param_grid = {  
    "max_depth": [3, 5, 7, 10],  
    "min_samples_split": [2, 5, 10]  
}
```

[47] ✓ 0.0s

```
grid = GridSearchCV(  
    DecisionTreeRegressor(random_state=42),  
    param_grid,  
    scoring="neg_root_mean_squared_error",  
    cv=5  
)
```

[26] ✓ 0.0s

```
grid.fit(X_train, y_train)
```

[27] ✓ 2.2s

...

- GridSearchCV ⓘ ?
- best_estimator_: DecisionTreeRegressor
 - DecisionTreeRegressor ?

```
print("Best Parameters:")  
print(grid.best_params_)
```

[48] ✓ 0.0s

... Best Parameters:
{'max_depth': 10, 'min_samples_split': 10}

AI_ML_Task3_Model_Validation_Tuning.ipynb > print("Best Parameters:")

Optimized RMSE : 0.6454300020019771

Optimized R2 : 0.6820992539714815

+ Code + Markdown | Run All Restart Clear All Outputs Jupyter Variables Outline ...

```
results = {
    "Model": [
        "Linear Regression",
        "Ridge Regression",
        "Tuned Decision Tree"
    ],
    "RMSE": [
        lr_rmse,
        ridge_rmse,
        rmse
    ],
    "R2 Score": [
        lr_r2,
        ridge_r2,
        r2
    ]
}

comparison = pd.DataFrame(results)

comparison
```

[32] ✓ 0.0s

	Model	RMSE	R2 Score
0	Linear Regression	0.745581	0.575788
1	Ridge Regression	0.745554	0.575819
2	Tuned Decision Tree	0.645430	0.682099

```
plt.figure(figsize=(8,5))

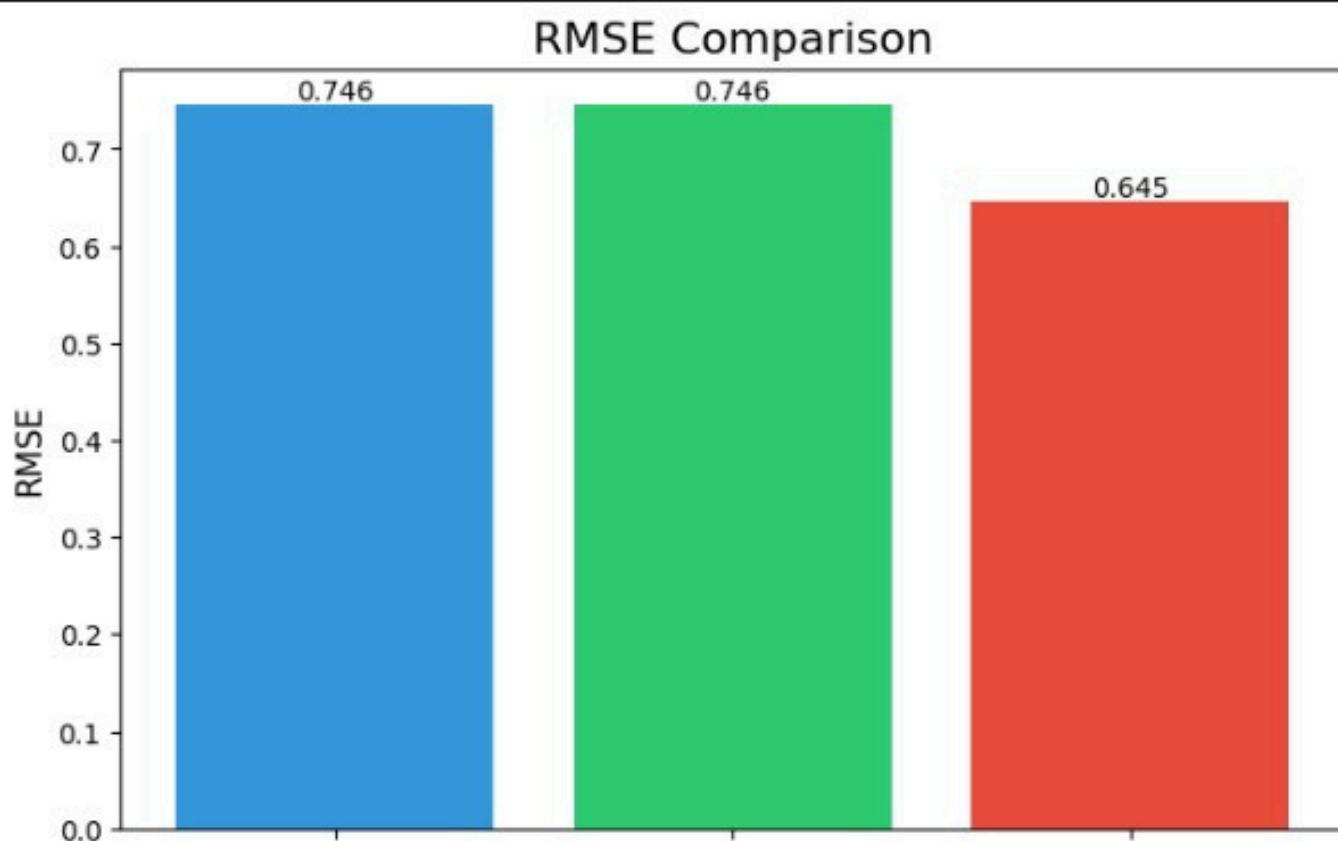
bars = plt.bar(
    comparison["Model"],
    comparison["RMSE"],
    color=["#3498db", "#2ecc71", "#e74c3c"]
)

plt.title("RMSE Comparison", fontsize=16)
plt.xlabel("Models", fontsize=12)
plt.ylabel("RMSE", fontsize=12)

for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width()/2,
        height,
        f"{height:.3f}",
        ha="center",
        va="bottom",
        fontsize=10
    )

plt.show()
```

[42] ✓ 0.0s




```
plt.figure(figsize=(7,5))

bars = plt.bar(
    ["Training RMSE", "Testing RMSE"],
    [train_rmse, test_rmse],
    color=["#3498db", "#f39c12"],
    edgecolor="black",
    linewidth=1.5
)

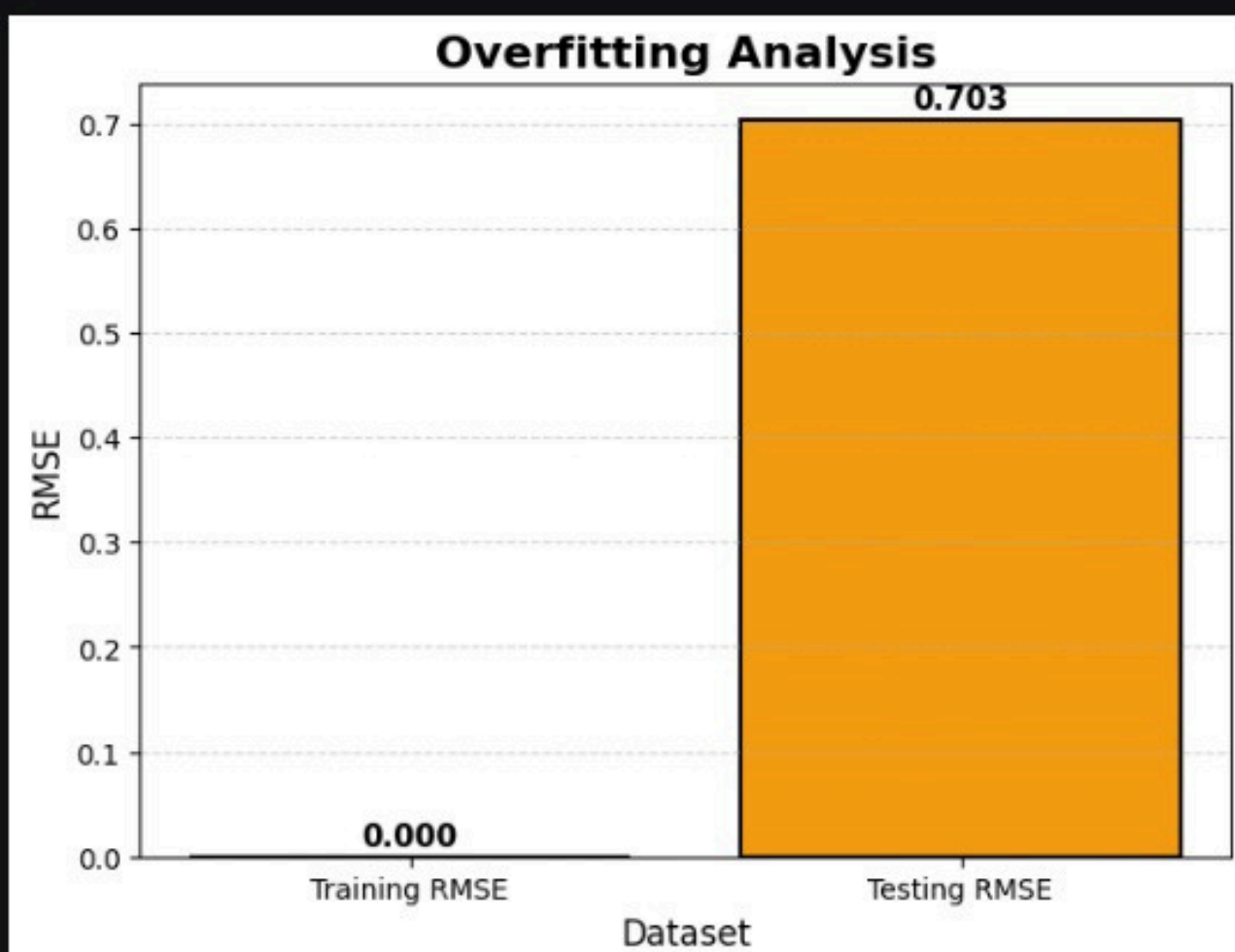
plt.title("Overfitting Analysis", fontsize=16, fontweight="bold")
plt.xlabel("Dataset", fontsize=12)
plt.ylabel("RMSE", fontsize=12)

for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width()/2,
        height + 0.01,
        f"{height:.3f}",
        ha="center",
        fontsize=11,
        fontweight="bold"
    )

plt.grid(axis="y", linestyle="--", alpha=0.5)

plt.show()
```

✓ 0.0s



AI_ML_Task3_Model_Validation_Tuning.ipynb > print("Best Parameters:")

Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ...

```
print("Project Conclusion")
print("-----")

print("1. Linear Regression model was trained successfully.")
print("2. Ridge Regression model was trained successfully.")
print("3. Decision Tree showed overfitting before tuning.")
print("4. Cross Validation gave reliable performance.")
print("5. GridSearchCV found the best parameters.")
print("6. Tuned Decision Tree improved model performance.")
print("7. Final model was selected based on RMSE and R2 Score.")
```

[37] ✓ 0.0s

... Project Conclusion

1. Linear Regression model was trained successfully.
2. Ridge Regression model was trained successfully.
3. Decision Tree showed overfitting before tuning.
4. Cross Validation gave reliable performance.
5. GridSearchCV found the best parameters.
6. Tuned Decision Tree improved model performance.
7. Final model was selected based on RMSE and R2 Score.

```
import joblib

joblib.dump(best_tree, "best_decision_tree_model.pkl")

print("Model saved successfully!")
```

[38] ✓ 0.0s

... Model saved successfully!